

SOURCE CODE

```
import tkinter as tk
from tkinter import Message ,Text
import cv2,os
import shutil
import csv
import numpy as np
from PIL import Image, ImageTk
import pandas as pd
import datetime
import time
import tkinter.ttk as ttk
import tkinter.font as font
#import serial_rx_tx
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)

from time import sleep, strftime, time
import _thread
import os
import time
import serial
import sys
from datetime import datetime
```

```
import datetime

buz=2

GPIO.setup(buz,GPIO.OUT)

GPIO.output(buz,1)

GPIO.setwarnings(False)

GPIO.setmode(GPIO.BCM)

ser = serial.Serial ("/dev/ttyS0")
```

```
def date_now():

    today = datetime.datetime.now().strftime("%Y-%m-%d")

    today = str(today)

    return(today)
```

```
def time_now():

    now = datetime.datetime.now().strftime("%H:%M:%S")

    now = str(now)

    return(now)
```

```
a1_h=16
```

```
a1_m=5
```

```
a2_h=16
```

```
a2_m=7
```

```
a3_h=16
```

```
a3_m=7
```

LCD_RS = 26

LCD_E = 19

LCD_D4 = 13

LCD_D5 = 6

LCD_D6 = 21

LCD_D7 = 20

GPIO.setup(LCD_E, GPIO.OUT) # E

GPIO.setup(LCD_RS, GPIO.OUT) # RS

GPIO.setup(LCD_D4, GPIO.OUT) # DB4

GPIO.setup(LCD_D5, GPIO.OUT) # DB5

GPIO.setup(LCD_D6, GPIO.OUT) # DB6

GPIO.setup(LCD_D7, GPIO.OUT) # DB7

Define some device constants

LCD_WIDTH = 16 # Maximum characters per line

LCD_CHR = True

LCD_CMD = False

LCD_LINE_1 = 0x80 # LCD RAM address for the 1st line

LCD_LINE_2 = 0xC0 # LCD RAM address for the 2nd line

Timing constants

E_PULSE = 0.0005

E_DELAY = 0.0005

```

def lcd_init():
    # Initialise display
    lcd_byte(0x33,LCD_CMD) # 110011 Initialise
    lcd_byte(0x32,LCD_CMD) # 110010 Initialise
    lcd_byte(0x06,LCD_CMD) # 000110 Cursor move direction
    lcd_byte(0x0C,LCD_CMD) # 001100 Display On,Cursor Off, Blink Off
    lcd_byte(0x28,LCD_CMD) # 101000 Data length, number of lines, font size
    lcd_byte(0x01,LCD_CMD) # 000001 Clear display
    time.sleep(E_DELAY)

```

```

def lcd_byte(bits, mode):
    # Send byte to data pins
    # bits = data
    # mode = True  for character
    #      False for command

    GPIO.output(LCD_RS, mode) # RS

    # High bits
    GPIO.output(LCD_D4, False)
    GPIO.output(LCD_D5, False)
    GPIO.output(LCD_D6, False)
    GPIO.output(LCD_D7, False)
    if bits&0x10==0x10:
        GPIO.output(LCD_D4, True)
    if bits&0x20==0x20:

```

```
GPIO.output(LCD_D5, True)
if bits&0x40==0x40:
    GPIO.output(LCD_D6, True)
if bits&0x80==0x80:
    GPIO.output(LCD_D7, True)
```

```
# Toggle 'Enable' pin
lcd_toggle_enable()
```

```
# Low bits
GPIO.output(LCD_D4, False)
GPIO.output(LCD_D5, False)
GPIO.output(LCD_D6, False)
GPIO.output(LCD_D7, False)
if bits&0x01==0x01:
    GPIO.output(LCD_D4, True)
if bits&0x02==0x02:
    GPIO.output(LCD_D5, True)
if bits&0x04==0x04:
    GPIO.output(LCD_D6, True)
if bits&0x08==0x08:
    GPIO.output(LCD_D7, True)
```

```
# Toggle 'Enable' pin
lcd_toggle_enable()
```

```

def lcd_toggle_enable():
    # Toggle enable
    time.sleep(E_DELAY)
    GPIO.output(LCD_E, True)
    time.sleep(E_PULSE)
    GPIO.output(LCD_E, False)
    time.sleep(E_DELAY)

def lcd_string(message,line):
    # Send string to display

    message = message.ljust(LCD_WIDTH," ")

    lcd_byte(line, LCD_CMD)

    for i in range(LCD_WIDTH):
        lcd_byte(ord(message[i]),LCD_CHR)


lcd_init()
lcd_byte(0x01,LCD_CMD)
lcd_string(" WELCOME",LCD_LINE_1)
##str_message=""
##serialPort = serial_rx_tx.SerialPort()
##

```

```

##

##def OpenCommand():
##
##    comport = 'COM4'
##    baudrate = '9600'
##    serialPort.Open(comport,baudrate)
##    #serialPort.Close()
##
##
def SendDataCommand(cmd):
    message = str(cmd)
    ser.write(message.encode())

import telepot

bot_token = '8570871199:AAFk0O5KLxTQIEoiA46xqUbppVqA-tgUEMk'
ser = serial.Serial ("/dev/ttyS0")
def send_csv_file(file_path, chat_id):
    bot = telepot.Bot(bot_token)
    bot.sendDocument(chat_id, open(file_path, 'rb'))

file_path = 'path/to/your/csv_file.csv'
chat_id = '7808941127' # repla202ce with the chat ID of the recipient

window = tk.Tk()
window.title("Face_Recogniser")

```

```
window.geometry('1280x720')
window.configure(background='white')
#window.attributes('-fullscreen', True)
window.grid_rowconfigure(0, weight=1)
window.grid_columnconfigure(0, weight=1)

slist=[1,2,3,4,5]
phlist=['9618393319','9392034450','9959213666','8328229605','9490506568']
```

```
message = tk.Label(window, text="FACE ATTENDANCE SYSTEM"
,bg="blue" ,fg="white" ,width=50 ,height=1,font=('times', 20, 'italic bold
underline'))
```

```
message.place(x=50, y=20)
```

```
lbl = tk.Label(window, text="Enter
ID",width=16 ,height=2 ,fg="black" ,bg="white" ,font=('times', 15, ' bold ') )
lbl.place(x=100, y=100)
```

```
txt = tk.Entry(window,width=16 ,bg="white" ,fg="black",font=('times', 15, '
bold '))
txt.place(x=400, y=115)
```

```
lbl2 = tk.Label(window, text="Enter
Name",width=16 ,fg="black" ,bg="white" ,height=2 ,font=('times', 15, ' bold
'))
lbl2.place(x=100, y=200)
```



```
txt2 = tk.Entry(window,width=16 ,bg="white" ,fg="black",font=('times', 15, '
bold ') )
```

```
txt2.place(x=400, y=215)
```

```
lbl3 = tk.Label(window, text="Notifcation :
",width=20 ,fg="orange" ,bg="white" ,height=2 ,font=('times', 15, ' bold
underline '))
```

```
lbl3.place(x=100, y=300)
```

```
message = tk.Label(window, text="....."
,bg="white" ,fg="black" ,width=30 ,height=2, activebackground = "yellow"
,font=('times', 15, ' bold '))
```

```
message.place(x=400, y=300)
```

```
lbl3 = tk.Label(window, text="Attendance :
",width=20 ,fg="black" ,bg="white" ,height=2 ,font=('times', 15, '
bold underline'))
```

```
lbl3.place(x=100, y=550)
```

```
message2 = tk.Label(window, text="....."
,fg="black" ,bg="white",activeforeground =
"green",width=30 ,height=2 ,font=('times', 15, ' bold '))
```

```
message2.place(x=400, y=550)
```

```
bot = telepot.Bot('6812948135:AAG6JbF2MnZnbkReoM8TmV2kaGebs-
zmTKI')
```

```
def handle(msg):
```

```
global telegramText
global chat_id
global receiveTelegramMessage
global inp
content_type, chat_type, chat_id = telepot.glance(msg)
print(content_type)
```

```
if content_type == 'text':
```

```
    chat_id = msg['chat']['id']
    telegramText = msg['text']
    print("Message received from " + str(chat_id))
    print("Message " + str(telegramText))
```

```
bot.message_loop(handle)
```

```
def send_sms(x):
```

```
    time.sleep(1)
```

```
    cmd='AT\r\n'
```

```
    SendDataCommand(cmd)
```

```
    time.sleep(2)
```

```
    cmd='AT+CMGF=1\r\n'
```

```
    SendDataCommand(cmd)
```

```
    time.sleep(2)
```

```
phno=phlist[x]
cmd='AT+CMGS="'+str(phno)+"\r\n'
SendDataCommand(cmd)
time.sleep(1)
cmd="Your child is absent today"
SendDataCommand(cmd) # Message
#ser.write(msg.encode()) # Message
time.sleep(1)
cmd = "\x1A"
SendDataCommand(cmd) # Enable to send SMS
time.sleep(5)
print('SMS Sent')
```

```
## global lat
## global lng
##
## print("sending SMS..")
## cmd='AT\r\n'
## ser.write(cmd.encode())
## time.sleep(2)
## rcv = ser.read(20)
## print(rcv)
## cmd='AT+CMGF=1\r\n'
## ser.write(cmd.encode())
## time.sleep(2)
## rcv = ser.read(20)
```

```

## print(rcv)
## phno=phlist[x]
## cmd='AT+CMGS="'+str(phno)+"\r\n'
## ser.write(cmd.encode())
## rcv = ser.read(20)
## print(rcv)
## time.sleep(1)
## cmd="The child is absent today"
## ser.write(cmd.encode()) # Message
##
## #ser.write(msg.encode()) # Message
## time.sleep(1)
## cmd = "\x1A"
## ser.write(cmd.encode()) # Enable to send SMS
## time.sleep(10)
## print('SMS Sent')
## time.sleep(1)

```

```

def clear():
    txt.delete(0, 'end')
    res = ""
    message.configure(text= res)

```

```

def clear2():
    txt2.delete(0, 'end')

```

```
res = ""  
message.configure(text= res)
```

```
def is_number(s):  
    try:  
        float(s)  
        return True  
    except ValueError:  
        pass  
  
    try:  
        import unicodedata  
        unicodedata.numeric(s)  
        return True  
    except (TypeError, ValueError):  
        pass  
  
    return False
```

```
def TakeImages():  
    Id=(txt.get())  
    name=(txt2.get())  
    if(is_number(Id) and name.isalpha()):  
        cam = cv2.VideoCapture(0)  
        harcascadePath = "haarcascade_frontalface_default.xml"  
        detector=cv2.CascadeClassifier(harcascadePath)
```

```

sampleNum=0
while(True):
    ret, img = cam.read()
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = detector.detectMultiScale(gray, 1.3, 5)
    for (x,y,w,h) in faces:
        cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)
        #incrementing sample number
        sampleNum=sampleNum+1
        #saving the captured face in the dataset folder TrainingImage
        cv2.imwrite("TrainingImage/"+name+"."+Id+"."+str(sampleNum) +
".jpg", gray[y:y+h,x:x+w])
        #display the frame
        cv2.imshow('frame',img)
        #wait for 100 milliseconds
        if cv2.waitKey(100) & 0xFF == ord('q'):
            break
        # break if the sample number is morethan 100
        elif sampleNum>60:
            break
    cam.release()
    cv2.destroyAllWindows()
    res = "Images Saved for ID : " + Id +" Name : "+ name
    row = [Id , name]
    with open('StudentDetails/StudentDetails.csv','a+') as csvFile:
        writer = csv.writer(csvFile)

```

```

        writer.writerow(row)

    csvFile.close()

    message.configure(text= res)
else:
    if(is_number(Id)):
        res = "Enter Alphabetical Name"

        message.configure(text= res)

    if(name.isalpha()):
        res = "Enter Numeric Id"

        message.configure(text= res)


def TrainImages():
    recognizer = cv2.face_LBPHFaceRecognizer.create()#recognizer =
cv2.face.LBPHFaceRecognizer_create()#$cv2.createLBPHFaceRecognizer()

    harcascadePath = "haarcascade_frontalface_default.xml"

    detector =cv2.CascadeClassifier(harcascadePath)

    faces,Id = getImagesAndLabels("TrainingImage")

    recognizer.train(faces, np.array(Id))

    recognizer.save("Trainer.yml")

    res = "Image Trained"#+" ".join(str(f) for f in Id)

    message.configure(text= res)


def getImagesAndLabels(path):
    #get the path of all the files in the folder

    imagePaths=[os.path.join(path,f) for f in os.listdir(path)]

    #print(imagePaths)

```

```

#create empty face list
faces=[]

#create empty ID list
Ids=[]

#now looping through all the image paths and loading the Ids and the images
for imagePath in imagePaths:

    #loading the image and converting it to gray scale
    pilImage=Image.open(imagePath).convert('L')

    #Now we are converting the PIL image into numpy array
    imageNp=np.array(pilImage,'uint8')

    #getting the Id from the image
    Id=int(os.path.split(imagePath)[-1].split(".")[1])

    # extract the face from the training image sample
    faces.append(imageNp)

    Ids.append(Id)

return faces,Ids

```

```

def TrackImages():

```

```

    global slist

```

```

    recognizer =
cv2.face.LBPHFaceRecognizer_create()#cv2.createLBPHFaceRecognizer()

    recognizer.read("Trainer.yml")

    harcascadePath = "haarcascade_frontalface_default.xml"

    faceCascade = cv2.CascadeClassifier(harcascadePath);

```



```

df=pd.read_csv("StudentDetails/StudentDetails.csv")
cam = cv2.VideoCapture(0)
font = cv2.FONT_HERSHEY_SIMPLEX
col_names = ['Id','Name','Date','Time']
attendance = pd.DataFrame(columns = col_names)
outtime=0
intime=0
while True:
    ret, im =cam.read()
    gray=cv2.cvtColor(im,cv2.COLOR_BGR2GRAY)
    faces=faceCascade.detectMultiScale(gray, 1.2,5)
    for(x,y,w,h) in faces:
        cv2.rectangle(im,(x,y),(x+w,y+h),(225,0,0),2)
        Id, conf = recognizer.predict(gray[y:y+h,x:x+w])
        if(conf < 70):
            ts = time.time()
            date = datetime.datetime.fromtimestamp(ts).strftime('%Y-%m-%d')
            timeStamp =
datetime.datetime.fromtimestamp(ts).strftime('%H:%M:%S')
            aa=df.loc[df['Id'] == Id]['Name'].values
            tt=str(Id)+"-"+aa
            hour = int(strftime("%H"))
            mint = int(strftime("%M"))
            sec = int(strftime("%S"))
            print (str(hour) + ":" + str(mint) + ":" + str(sec))
            sleep(1)

```

```

if(hour == a1_h and mint == a1_m and sec<10):
    intime=1
    print('starting')
    GPIO.output(buz,0)
    time.sleep(1)
    GPIO.output(buz,1)
if(hour == a2_h and mint == a2_m and sec<10):
    intime=0
    print('ending')
    GPIO.output(buz,0)
    time.sleep(1)
    GPIO.output(buz,1)
if(hour == a3_h and mint == a3_m and sec<10):
    outtime=1

if(intime==1):
    attendance.loc[len(attendance)] = [Id,aa,date,timeStamp]
if(Id in slist):
    slist.remove(Id)
print(slist)
# lcd_byte(0x01,LCD_CMD)
# lcd_string("S:" + str(attendance),LCD_LINE_1)
#lcd_string("T:" + str(tval),LCD_LINE_2)

else:

```

```

        Id='Unknown'

        tt=str(Id)

        if(conf > 75):

            noOfFile=len(os.listdir("ImagesUnknown"))+1

            cv2.imwrite("ImagesUnknown/Image"+str(noOfFile) + ".jpg",
im[y:y+h,x:x+w])

            cv2.putText(im,str(tt),(x,y+h), font, 1,(255,255,255),2)

            lcd_byte(0x01,LCD_CMD)

            lcd_string(str(tt),LCD_LINE_1)

            attendance=attendance.drop_duplicates(subset=['Id'],keep='first')

            cv2.imshow('im',im)

            if (cv2.waitKey(1)==ord('q') or outtime==1):

                break

            print('sending info')

            ts = time.time()

            date = datetime.datetime.fromtimestamp(ts).strftime('%Y-%m-%d')

            timeStamp = datetime.datetime.fromtimestamp(ts).strftime('%H:%M:%S')

            Hour,Minute,Second=timeStamp.split(":")

            fileName="Attendance/Attendance_" +date+"_" +Hour+"-"+Minute+"-
"+Second+".csv"

            attendance.to_csv(fileName,index=False)

            cam.release()

            cv2.destroyAllWindows()

            #print(attendance)

            res=attendance

            message2.configure(text= res)

            send_csv_file(fileName, chat_id)

```

```
for x in range(len(slist)):
```

```
    print ('Sending SMS To'+ str(slist[x]))
```

```
    send_sms(slist[x])
```

```
    time.sleep(0.1)
```

```
clearButton = tk.Button(window, text="Clear",  
command=clear ,fg="black" ,bg="gray" ,width=10 ,height=1  
,activebackground = "Red" ,font=('times', 15, ' bold '))
```

```
clearButton.place(x=650, y=100)
```

```
clearButton2 = tk.Button(window, text="Clear",  
command=clear2 ,fg="black" ,bg="gray" ,width=10 ,height=1,  
activebackground = "Red" ,font=('times', 15, ' bold '))
```

```
clearButton2.place(x=650, y=200)
```

```
takeImg = tk.Button(window, text="Take Images",  
command=TakeImages ,fg="black" ,bg="gray" ,width=10 ,height=2,  
activebackground = "Red" ,font=('times', 15, ' bold '))
```

```
takeImg.place(x=100, y=400)
```

```
trainImg = tk.Button(window, text="Train Images",  
command=TrainImages ,fg="black" ,bg="gray" ,width=10 ,height=2,  
activebackground = "Red" ,font=('times', 15, ' bold '))
```

```
trainImg.place(x=300, y=400)
```

```
trackImg = tk.Button(window, text="Track Images",  
command=TrackImages ,fg="black" ,bg="gray" ,width=10 ,height=2,  
activebackground = "Red" ,font=('times', 15, ' bold '))
```

```
trackImg.place(x=500, y=400)
```

```
quitWindow = tk.Button(window, text="Quit",  
command=window.destroy ,fg="black" ,bg="gray" ,width=10 ,height=2,  
activebackground = "Red" ,font=('times', 15, ' bold '))  
quitWindow.place(x=700, y=400)
```

```
window.mainloop()
```